



中山大學
SUN YAT-SEN UNIVERSITY

本科生课程报告

软件配置与运维文档

寻迹校园失物招领平台

课程名称： 软件工程

项目名称： 寻迹校园失物招领平台

团队名称： 寻迹项目组

项目负责人： 朱梓涵

团队成员： 朱梓涵、周星辰、林洪宽、郭焜华、程文韬

负责人学号： 24325356

文档版本： V1.0

基线日期： 2026 年 7 月 12 日

中山大学

2026 年 7 月

目录

1	文档概述	1
2	配置管理	1
3	版本控制与数据库版本	3
4	Docker 部署架构	5
5	持续集成与交付	7
6	部署与变更操作	8
7	健康检查、日志与监控	10
8	备份、恢复与回滚	10
9	安全运维	12
10	应急响应计划	13
11	持续优化与扩展路线	14
12	结论	15

1 文档概述

1.1 目的与适用范围

本文件描述「寻迹」校园失物招领平台的配置管理、版本控制、持续集成、容器部署、运行监测、备份恢复、回滚、安全与应急计划。

本文是截至 2026 年 7 月 12 日的课程期末成果版。系统面向校内课程演示，已形成环境变量模板、启动安全校验、Git/Alembic 版本化、六服务 Compose、持久卷、健康检查、持续集成与部署/应急操作规范。互联网规模的高可用、自动发布和集中观测统一放入「持续优化与扩展路线」。

表 1: 配置与运维成果摘要

领域	期末成果	阶段
配置管理	使用环境变量、Pydantic Settings 和示例文件；真实 .env 被 Git 与镜像上下文排除	已完成
版本控制	约定 main/develop/feature/fix，使用 Alembic 和本标签，文本行尾统一为 LF	已完成
持续集成	五个并行 job 覆盖静态检查、测试、构建、Compose 配置和两个 Python 镜像	已验证
容器部署	Compose 编排 MySQL、MinIO、AI、Backend、用 户端和管理端六服务	已完成
运行保障	健康端点、Loguru、业务操作日志、持久卷和部署后核验流程	已完成
生产扩展	不可变镜像、自动发布、集中观测和备份恢复自动化	持续优化

2 配置管理

2.1 配置原则

项目遵循「代码、非敏感默认值、敏感配置分离」的原则。Backend 和 AI 服务通过 Pydantic Settings 从进程环境及可选 .env 读取配置；Compose 从 deploy/docker/.env 注入环境变量；前端 API 基址在构建阶段通过 VITE_API_BASE_URL 固化。示例配置可以进入仓库，真实密钥、数据库口令和本地覆盖文件不得提交。

根目录 .gitignore 忽略 .env 与 .env.*，只放行 .env.example；.dockerignore 同时排除各层真实环境文件，防止密钥进入 Docker build context。凭据遵循定期轮换

策略；一旦进入 Git 历史、日志或镜像，必须立即在对应平台撤销并更换。

2.2 配置项分类

表 2:主要配置项及约束

类别	主要变量	约束与用途	完成情况
数据库	DB_HOST、DB_PORT、DB_NAME、DB_USER、DB_PASSWORD	MySQL 8.4 连接；Compose 中库名、用户和密码为必填	已完成
鉴权	JWT_SECRET_KEY、JWT_EXPIRE_DAYS	非 Debug 环境使用默认 JWT 密钥时拒绝启动	已完成
管理员初始化	BOOTSTRAP_ADMIN_ENABLED、账号、密码、手机号	只在显式开启且账号不存在时创建；创建后立即关闭开关	已完成
短信演示	DEBUG、SMS_DEBUG_ENABLED、SMS_DEMO_PHONES	三项同时满足才向精确白名单返回调试码；不支持通配符	已完成
对象存储	Endpoint、access key、secret key、bucket、region、签名时长	bucket 应保持私有；普通、敏感和 AI 内部签名采用不同有效期	已完成
AI 内部鉴权	AI_SERVICE_BASE_URL、AI_SERVICE_TOKEN、timeout	共享环境 token 至少 32 字符；业务路由要求 X-Service-Token	已完成
模型供应商	API key、base URL、文本/向量/视觉模型名	key 为空时使用规则基线；模型失败不得阻断基础发布	已完成
URL 安全	公开图片 host 白名单、可信私有 host 精确列表	私网 IP 和单标签主机默认拒绝；MinIO 仅按精确 host 例外	已完成
匹配任务	自动间隔、并发数、最大候选数	自动间隔为 0 时关闭；管理员可手工触发	已完成
Durable job	轮询、最大尝试、退避、锁超时、敏感任务并发	任务持久化、失败退避、遗留 RUNNING 回收	已完成
CORS	CORS_ALLOWED_ORIGINS	必须为显式来源列表，通配符被启动校验拒绝	已完成

表 2 (续)

类别	主要变量	约束与用途	完成情况
前端 API	VITE_API_BASE_URL	完整栈使用相对 /api/v1 经 Nginx 代理；单独构建可指定绝对地址	已完成

2.3 环境划分

表 3: 环境与变更约束

环境	用途	配置要求	数据策略
本地开发	单人编码、自测	可使用规则 AI；允许显式 Debug 和受控短信白名单	可重建，禁止使用真实敏感数据
小组联调	API 与页面集成	固定地址、非默认密钥、共享 AI token、关闭任意调试入口	固定测试数据，定期清理
演示环境	课程答辩	冻结提交和迁移，尽量复用已验收联调栈，准备离线降级	账号和数据稳定，演示前备份
生产扩展	高可用与安全加固	HTTPS、外部 secret、最小权限、监控、备份恢复和真实短信	脱敏测试副本，迁移先演练

课程交付使用可重复的本地/演示型 Compose，docker-compose.production.yml 用于隐藏 MySQL 和 MinIO 主机端口。TLS 网关、secret store 与集中观测属于生产扩展路线。

3 版本控制与数据库版本

3.1 Git 分支与提交规范

仓库约定 main 为可发布分支、develop 为集成分支，功能和修复分别使用 feature/<name> 与 fix/<name>。提交信息采用简短 Conventional Commit 风格，API、

枚举、数据库和流程变化必须同步文档；数据库结构变化必须附 Alembic 迁移。

仓库使用 v1.0.0-core 标记核心闭环基线，并通过 .gitattributes 规定普通文本使用 LF、Windows 批处理使用 CRLF，图片、PDF、字体和压缩包按 binary 处理。

表 4: 版本控制要素

要素	实现方式	完成情况
分支模型	main/develop/feature/fix 写入贡献指南	已完成
提交规范	feat、fix、test、docs、chore 等短提交；可关联 FR	已完成
发布标记	v1.0.0-core 标识核心闭环基线	已完成
行尾与二进制	.gitattributes 统一跨平台规则	已完成
Pull Request	feature 到 develop 的协作与评审记录	已完成

required status checks、非作者评审、禁止强推和以 commit SHA 标记不可变镜像，可作为生产化协作与发布治理的持续扩展。

3.2 Alembic 迁移管理

数据库以 SQLAlchemy 模型和 Alembic 历史为真相源，不再使用手写 backend/sql/schema.sql 初始化新环境。Backend 容器启动命令先执行 alembic upgrade head，成功后再启动 Uvicorn。当前迁移形成单一 head 20260711_0008。

表 5: 数据库迁移序列

Revision	主要内容	说明
20260428_0001	用户、认证、失物、招领、图片、验证问题和操作日志基础表	Backend A 基线
20260428_0002	匹配、认领、答案、交接、积分、通知、举报和公告	Backend B 基线
20260428_0003	失物与招领内容审核状态及索引	治理扩展
20260711_0004	核心状态机字段、匹配 pair 唯一约束及重复引用迁移	历史数据映射
20260711_0005	内容审核意见字段	契约扩展
20260711_0006	匹配分数来源、降级和图像可用性	可解释性

表 5 (续)

Revision	主要内容	说明
20260711_0007	举报目标去重约束及一致性处理	去重后建立唯一约束
20260711_0008	durable job/outbox 表、索引和约束	当前 head

所有迁移都定义 downgrade。生产变更必须先和数据副本演练、生成核对报告并备份，优先使用向前修复；0004 与 0007 涉及历史数据整理，禁止把 alembic downgrade 当作无损撤销按钮。

4 Docker 部署架构

4.1 六服务拓扑

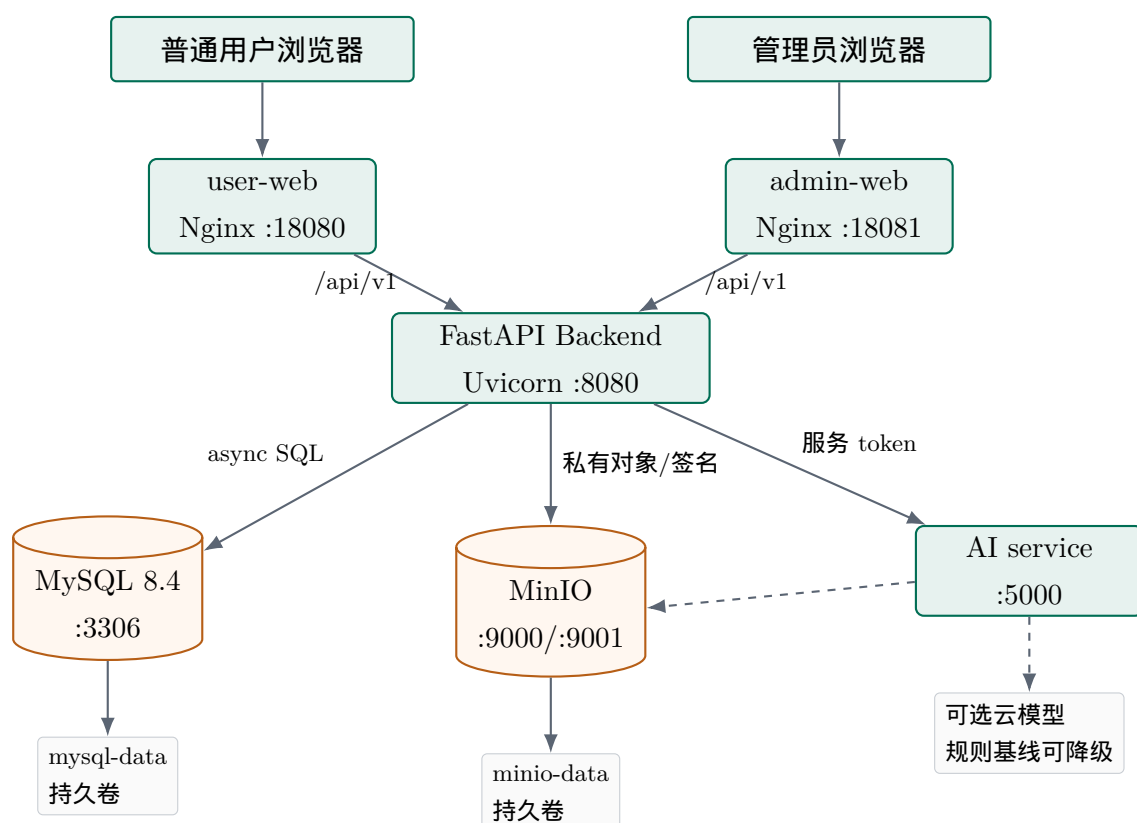


图 1: 本地完整栈部署拓扑

完整栈中两个前端以 Nginx 提供静态文件并代理 /api/v1；Backend 负责业务、迁移和对象签名；AI 服务是独立受鉴权辅助服务；MySQL 和 MinIO 使用命名卷持久化。

AI 不可用时，分类和匹配可退回规则基线，基础发布不能因模型故障被回滚。

4.2 服务清单与依赖

表 6: Compose服务清单

服务	镜像/构建	主机端口	持久化与健康	启动条件
mysql	mysql:8.4	默认仅 127.0.0.1:3306	mysql-data ; mysqladmin healthcheck	root 和业务口令 必填
minio	固定日期 release tag	仅本机端口 9000、9001	minio-data 持久卷	访问凭据必填
ai-service	本仓库 Dockerfile	默认仅 127.0.0.1:5000	无状态服务；HTTP liveness	规则基线可离线运 行；服务 token 必 填
backend	本仓库 Dockerfile	0.0.0.0:8080	HTTP liveness；启 动前迁移	等待 MySQL healthy , AI/MinIO started
user-web	Node 构建、 Nginx 运行	18080:80	无状态静态服务	依赖 Backend started
admin-web	Node 构建、 Nginx 运行	18081:80	无状态静态服务	依赖 Backend started

所有服务配置 `restart: unless-stopped`。MySQL、AI 和 Backend 提供健康检查，首次启动或依赖抖动时按部署后清单执行重试与人工核验。数据库、MinIO、AI 组合 `readiness` 与前端探针可在生产观测扩展中统一引入。

4.3 Compose 变体

表 7: 部署文件用途

文件	用途与差异	状态
<code>docker-compose.yml</code>	六服务本地栈，回环暴露 MySQL/MinIO/AI，Backend 和前端可从主机访问	课程主配置
<code>docker-compose.frontend.yml</code>	单独构建两个前端，API 指向外部 Backend	联调配置
<code>docker-compose.podman-host.yml</code>	rootless Podman DNS 场景使用 host network 适配	环境适配
<code>docker-compose.production.yml</code>	叠加后移除 MySQL 和 MinIO 主机端口	生产扩展

课程演示使用本地主配置；公网环境在生产 overlay 之上继续扩展 HTTPS 网关、统一对象访问域名、最小暴露面和不可变镜像。

5 持续集成与交付

5.1 CI 工作流

GitHub Actions 在 main/develop 的 push 和 pull request 上触发，并通过 concurrency 取消同一引用的旧运行。最近一次绿色运行 29162106877 对应 708f011，各 job 如下。

表 8: CI Job 与最近结果

Job	步骤	最近结果
Backend	uv 同步；Ruff lint/format；严格 Mypy；Pytest coverage；上传 XML	200 passed，80% 覆盖率，80 个源文件类型通过
AI service	uv 同步；Ruff lint/format；严格 Mypy；Pytest	60 passed，14 个源文件类型通过

表 8 (续)

Job	步骤	最近结果
Frontend user-app	pnpm 冻结安装 ; vue-tsc ; Vitest ; Vite build	17 passed , 构建通过
Frontend admin-web	pnpm 冻结安装 ; vue-tsc ; Vitest ; Vite build	15 passed , 构建通过
Docker build smoke	Compose config ; 构建 Backend 与 AI 镜像	全部成功 , 未启动容器

工作流采用锁文件安装、四模块并行、静态与运行门禁组合，并上传 Backend 覆盖率 XML。MySQL/MinIO 集成、六服务 smoke、浏览器 E2E、多模块覆盖率与依赖版本升级归入 CI 持续扩展。

5.2 持续集成成果与交付扩展

课程交付已完成代码校验、测试、前端生产构建、Compose 配置和 Backend/AI 镜像构建的持续集成闭环。

持续交付扩展可在独立环境中以完整 SHA 和语义版本标记镜像，经人工批准后发布；发布前执行备份与迁移演练，发布后执行 readiness 和 P0 smoke，异常时回退应用镜像并按迁移策略恢复或向前修复数据库。

6 部署与变更操作

6.1 部署前检查

部署者应确认 Docker Compose 或 Podman Compose 可用、端口无冲突、磁盘容量足够，并从示例文件生成本地配置。所有空白密钥必须填入不同的随机值；共享环境禁止使用示例 token、默认管理员密码或通配 CORS。还应确认目标数据库是新库还是历史 schema.sql 库，后者不能直接 stamp head。

本地完整栈启动示例

```
cp deploy/docker/.env.example deploy/docker/.env
# Edit deploy/docker/.env and fill every required secret.
docker compose --env-file deploy/docker/.env \
  -f deploy/docker/docker-compose.yml config --quiet
```

```
docker compose --env-file deploy/docker/.env \  
-f deploy/docker/docker-compose.yml up --build -d  
docker compose --env-file deploy/docker/.env \  
-f deploy/docker/docker-compose.yml ps
```

Backend 容器会先执行 `alembic upgrade head`。若管理员尚不存在，可在首次部署显式设置 `BOOTSTRAP_ADMIN_ENABLED=true`；创建完成并验证登录后立即恢复为 `false`。演示短信同样只在本地、精确白名单和双重 Debug 开关下启用。

6.2 部署后核验

1. 检查六服务状态和最近日志，确认 Backend 迁移完成且没有重启循环。
2. 访问 Backend 与 `AI /health`，再实际查询数据库支持的只读接口。
3. 验证用户端 18080、管理端 18081 以及 Nginx `/api/v1` 代理。
4. 验证 MinIO bucket 没有匿名策略；普通和敏感资产的签名时长及角色权限正确。
5. 使用演示账号执行登录、发布、搜索、匹配、认领和管理端审核的最小 smoke。
6. 记录提交 SHA、镜像摘要、Alembic revision、配置项名称摘要和验证结果，禁止记录秘密值。

课程演示完成后如需清理本地数据，可执行 `docker compose down -v`，但该命令会删除命名卷，属于破坏性重置，不是备份、恢复或正常回滚手段。

7 健康检查、日志与监控

7.1 健康与日志成果

表 9: 运行观测能力

能力	实现方式	完成情况
进程健康	MySQL ping、AI HTTP health、Backend HTTP health	已完成
应用日志	Backend、AI 和 durable job 使用 Loguru 输出启动、异常、降级和重试	已完成
业务审计	管理员认证、内容、举报、用户等关键操作写 operation log	已完成
请求关联	异常响应生成或接收 requestId，方便问题定位	已完成

7.2 持续优化：监控与告警

生产观测可扩展 HTTP 请求数、延迟分位、4xx/5xx 和业务冲突率；数据库连接池、慢查询和锁等待；MinIO 容量、错误和签名失败；AI 调用延迟、超时、降级率和费用；durable job 的 PENDING/RUNNING/FAILED 数、重试次数和最老任务年龄，以及容器的 CPU、内存和磁盘指标。

建议设置三级告警：P0 为敏感资产公开、数据库不可用、迁移失败和主流程完全中断；P1 为错误率或任务积压持续超阈值、MinIO 不可用和 AI 大面积超时；P2 为容量趋势、单接口性能和非关键页面异常。告警必须包含环境、版本、requestId、时间范围和处置手册链接，但不得包含密码、OTP、token 或原始证件数据。

8 备份、恢复与回滚

8.1 课程持久化与生产扩展

MySQL 和 MinIO 命名卷完成课程演示环境的本机持久化。生产扩展采用离机、加密、可校验的备份方案，并通过隔离环境恢复演练确认 RPO/RTO。

表 10: 生产扩展备份方案

对象	方法	频率与保留	校验/恢复目标	阶段
MySQL	一致性逻辑备份；生产规模扩大后使用物理备份与 binlog	每日；保留 7 日，周备份保留 4 周	校验可导入；目标 RPO $\leq$ 24 h	生产扩展
MinIO	使用管理客户端镜像/同步到独立存储，保留对象版本与清单	每日增量、每周全量	随机抽样 hash；对象与 DB 引用对账	生产扩展
配置	保存无秘密模板、变量名清单和加密 secret 备份	每次变更	可在新主机重建，秘密双人授权	生产扩展
镜像与源码	commit SHA 镜像标签、Git tag、锁文件与迁移历史	每次发布	可取得上一绿色版本的不可变镜像	生产扩展
演练记录	隔离环境执行全量恢复和 P0 smoke	每学期/重大迁移前	目标 RTO $\leq$ 4 h (课程环境)	生产扩展

备份必须离开运行主机并加密保存；至少一份副本不可被日常应用凭据删除。每次备份记录开始/结束时间、版本、大小、校验值和执行者。只有实际完成恢复并通过迁移、对象引用和 P0 smoke 的备份，才能称为「可恢复」。

8.2 恢复流程

1. 宣布维护窗口，冻结写入并记录故障版本、数据库 revision 和对象清单。
2. 在隔离环境创建空 MySQL 与空 MinIO，验证备份文件和密钥可读。
3. 先恢复数据库和对象，再核对所有 asset:// 引用所对应对象是否存在。
4. 使用与备份一致的应用版本启动，确认 revision；按批准方案迁移到目标 head。
5. 验证管理员登录、普通用户读取、敏感资产权限、发布和交接最小路径。
6. 经负责人确认后切换流量，保留旧环境只读直至观察期结束。

8.3 应用与数据库回滚

生产扩展中，应用回滚切换到上一绿色 commit 对应的不可变镜像，禁止重新构建同名:local。每次发布记录镜像 digest，并保持新旧版本的配置兼容窗口。

数据库默认采用「备份 + 向前修复」。对于只增加兼容字段的迁移，可先部署扩展结构，再部署应用；删除或收紧约束使用 expand-migrate-contract。Alembic downgrade 只在隔离副本验证无数据损失后使用。若迁移已经改写或删除数据，应从部署前备份恢复或编写专用修复脚本，不能盲目 downgrade/stamp。

9 安全运维

表 11: 安全控制与扩展方向

领域	课程版控制	生产扩展
密钥	Compose 对主要秘密使用必填表达式；非 Debug 默认 JWT/管理员密码拒绝启动	Docker/Podman secrets、定期轮换、泄露响应和 secret scanning
网络	MySQL、MinIO、AI 默认仅绑定回环；Production overlay 隐藏 DB/对象端口	统一 HTTPS 网关，只公开 443，限制 Backend 管理面
对象存储	bucket 私有、短期签名、敏感签名更短、AI 内部签名分离	MinIO root 与应用服务账号拆分，匿名拒绝自动验收
服务鉴权	AI 业务接口要求至少 32 字符 token；CORS 显式白名单	token 双向轮换、mTLS 或网络策略
容器	构建使用冻结依赖锁，镜像构建进入 CI	非 root、只读 rootfs、cap_drop、digest、SBOM 和漏洞扫描
日志隐私	关键操作写入 operation log	字段白名单、集中脱敏、轮转和保留期策略
账号与短信	调试验证码需 Debug、开关与精确号码三条件	真实 SMS Provider、Redis/DB OTP、IP/设备限流

凭据泄露时应立即撤销第三方 key、数据库用户、MinIO key、JWT 和 AI token；检查 Git、CI artifact、日志和镜像是否包含旧值；重新部署后验证旧凭据失效。仅把文

件加入 ignore 不能完成处置。

10 应急响应计划

10.1 事件分级与通用流程

表 12: 运行事件分级

等级	示例	响应要求
S0	敏感原图公开、管理员接管、数据库破坏、全站不可用	立即隔离、停止写入、轮换凭据、保全日志与现场记录并通知负责人
S1	登录/发布/认领主链路不可用，任务大量失败，MinIO 故障	30 分钟内确认影响，启用降级或回退，持续记录时间线
S2	单页面、单模型或非关键统计异常	建立缺陷，提供绕行，进入最近维护窗口

通用处置顺序为：发现与确认、限制影响、保留日志和版本记录、选择降级/回退/恢复、验证数据一致性、恢复服务、复盘并增加监控和回归用例。应急期间禁止直接在生产数据库手工修改而不留 SQL、执行者和审批记录。

10.2 典型场景处置

表 13: 典型故障处置卡

场景	立即处置	恢复与复盘
数据库不可用	暂停写流量；检查容器、磁盘、连接和最近迁移；禁止反复重启掩盖日志	恢复副本或服务；核对 revision 和事务；执行 P0 smoke，补充容量/连接告警
迁移失败	保留失败日志和部署前备份；阻止新应用启动；不要直接 stamp	在副本重现；修正迁移或恢复备份；核对约束与引用后再发布

表 13 (续)

场景	立即处置	恢复与复盘
MinIO 不可用/公开	暂停上传和签名；如公开则立即移除匿名 policy、轮换 key	DB 与对象清单对账；验证各角色访问；补充健康检查和匿名探测
AI 超时或供应商故障	保持规则基线和 fail-closed；限制重试避免放大费用与锁等待	检查 timeout、降级率和队列；供应商恢复后小流量验证，不回写过期结果
durable job 积压	暂停新增高成本任务或降低并发；检查 FAILED、锁超时与外部依赖	安全重试前核对业务版本；补充积压指标和管理员重试入口
凭据泄露	撤销/轮换相关 key；隔离受影响服务；保存访问日志	搜索 Git、artifact、日志和镜像；验证旧凭据失效并复盘泄露路径
前端发布失败	保持上一静态版本；检查 API 基址、Nginx upstream 和缓存	使用上一 digest；清理错误缓存；验证登录和 API 代理

11 持续优化与扩展路线

1. **集成环境**：在 CI 中启动六服务，执行 MySQL/MinIO 集成、健康等待与浏览器 smoke。
2. **持续交付**：以 commit SHA 与语义版本标记镜像，增加环境审批、发布后校验与自动回退。
3. **安全入口**：引入统一 HTTPS 网关、私有网络、对象访问域名与最小暴露面。
4. **备份恢复**：实现离机加密备份、校验、恢复脚本与定期演练，持续校准 RPO/RTO。
5. **集中观测**：增加结构化日志、指标、告警、trace、轮转、容量与 durable job 运维面板。
6. **容器供应链**：采用非 root、只读文件系统、镜像 digest、SBOM 和漏洞扫描。
7. **迁移演练**：在 MySQL 脱敏数据副本上执行 upgrade/downgrade，对 0004 与 0007 进行专项数据核对。

扩展顺序为：凭据轮换、HTTPS/私有网络、MySQL/MinIO 集成与备份恢复优先；随后建立不可变镜像与审批式持续交付；再扩展 readiness、集中日志、指标告警和任务运维入口；最后完成非 root、只读文件系统、SBOM、漏洞扫描与容量优化。

12 结论

项目已完成课程期末要求的配置与运维工程：环境变量模板、启动安全校验、Git/Alembic版本化、六服务 Compose、持久卷、健康与日志能力、部署/回滚/应急规范，以及覆盖四个代码模块的持续集成。最近远端 CI 中 Backend 200 项、AI Service 60 项、用户端 17 项和管理端 15 项测试全部通过，双前端生产构建与 Backend/AI 镜像构建同步通过。

课程版已实现可复现演示部署、可追溯数据库迁移、自动化质量校验和成体系的运维操作说明。持续优化将进一步扩展 HTTPS、最小权限 secret、真实依赖集成、集中观测、备份恢复演练和不可变回滚，使现有成果可平滑演进到更高规模的运行环境。